Numerical and simulation techniques in finance

Nicolas Langrené

Beijing Normal-Hong Kong Baptist University

2025 Fall Semester

2 Numerical computation

Computer arithmetic

Two sources of error occur systematically in numerical computation:

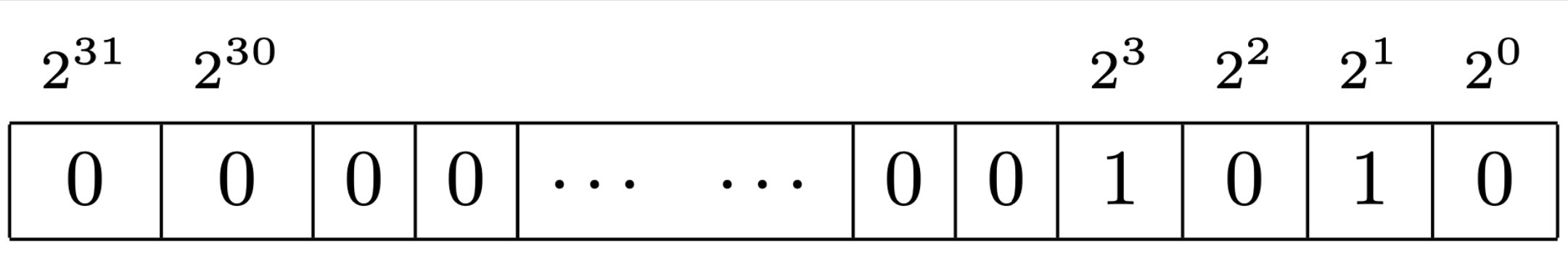
▷ **Truncation errors** due to the simplification of a mathematical model (e.g. discretization, replacement of a derivative by a finite difference)

▷ **Rounding errors** due to the fact that it is impossible to represent real numbers exactly in a computer.

2.1 Integer arithmetic

Integer arithmetic

The support for all information in a computer is a “word” constituted by a sequence of bits (generally 64), a bit having value 0 or 1.



This sequence of bits is then interpreted as the binary representation of an unsigned integer. As a consequence integers can be represented ***exactly*** in a computer. If computations only involve integers, they follow the rules of **integer arithmetic** (arithmetic without fractions).
Such computations are ***exact***.

Signed integers

To represent negative integers in binary form, the **two's complement method** is used:

	Value = -41	Value = -44
1. Starting from the right, find the first '1'	00101100**1**	00101**1**00
2. Invert all the bits to the left of that one	**11010011**1	**11010**100

Representing negative numbers in this way makes it possible to compute the sum of two numbers, regardless of their signs, via normal binary addition:

5	00000101	-5	11111011
+(-3)	11111101	+(-4)	11111100
------	----------	------	----------
2	00000010	-9	11110111

Python integers

In Python integers can be arbitrarily large. The interpreter simply uses as many bits as needed to represent the number.

```
one = 1
one.bit_length()
## 1
googol = 10 ** 100
googol.bit_length()
## 333
myrillion = 10 ** 30003
myrillion.bit_length()
## 99668
```

```python
import textwrap
myrillion  = 10**30003   # 1 followed by 30003 zeros
list_to_print = textwrap.wrap(str( myrillion ), 138)   # print 138 zeros per line
print(str(len(list_to_print)) + ' lines of zeros to print')
## 218 lines of zeros to print
for (i,s) in enumerate(list_to_print):
    print('['+str(i).zfill(2)+']  '+s)
## [00]  10000000000
```

NumPy integers

Unfortunately, numerical packages such as NumPy are not compatible with this extensible integer type https://numpy.org/doc/stable/user/basics.types.html

```
print(np.iinfo(np.int32))
## Machine parameters for int32
## ---------------------------------------------------------------
## min = -2147483648
## max = 2147483647
## ---------------------------------------------------------------
print(np.iinfo(np.int64))
## Machine parameters for int64
## ---------------------------------------------------------------
## min = -9223372036854775808
## max = 9223372036854775807
```

NumPy integers

NumPy uses a fixed number of bits to store integers, usually **32 bits** (np.int32) or **64 bits** (np.int64, default). Computations outside the maximum range of values generate overflows (truncation of the bits outside the fixed range).

```
v = np.array([10,100])
v[0]**v[1]   # overflow
## 0
v = np.array([99,99])
v[0]**v[1]   # overflow
## -7394533151961528133
```

⚠ 32 bits might not be enough for financial applications
Example: number of paths for accurate Monte Carlo estimation

2.2 Floating-point arithmetic

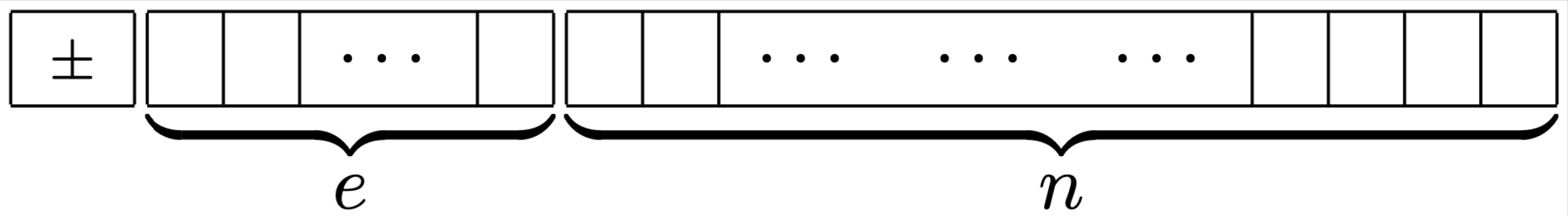
Representation of real numbers

In a computer, a real number $\mathbf{x} \neq 0$ is represented as $\mathbf{x} = \pm \mathbf{n} \times \mathbf{b}^{\mathbf{e}}$

- **n** is the mantissa / significand / fractional part
- **b** the base (always 2)
- **e** the exponent

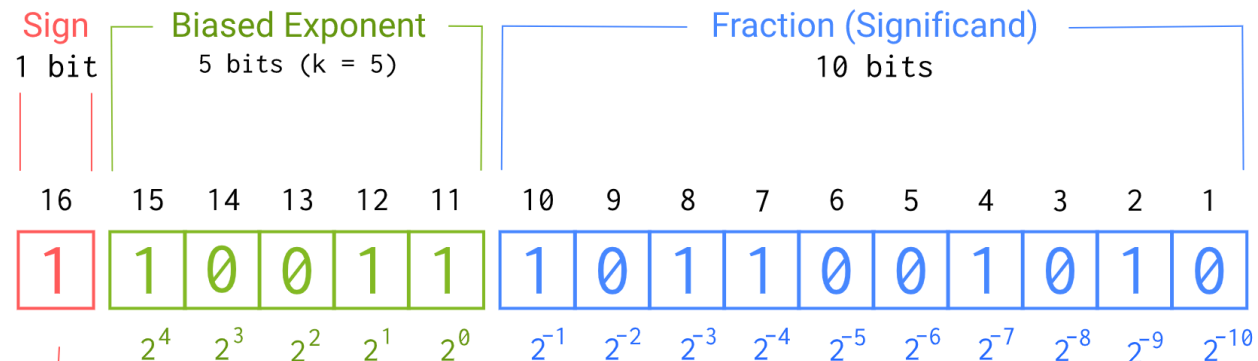
Example: $\mathbf{21.046875 = 1.3154296875 \times 2^4 = 1347 \times 2^{-6}}$

In practice the word is partitioned into three parts: the first bit indicates the sign, the second piece stores the exponent **e** and the final and largest piece stores **n**.



Half-precision 16 bits

trekhleb.dev



$= -27.15625$

$biased_exponent = 2^4 + 2^1 + 2^0 = 19$

$bias = 2^{k-1} - 1 = 2^{5-1} - 1 = 15$

$exponent = biased_exponent - bias = 19 - 15 = 4$

$fraction = 2^{-1} + 2^{-3} + 2^{-4} + 2^{-7} + 2^{-9} =$

$= 0.5 + 0.125 + 0.0625 + 0.0078125 + 0.001953125 =$

$= 0.697265625$

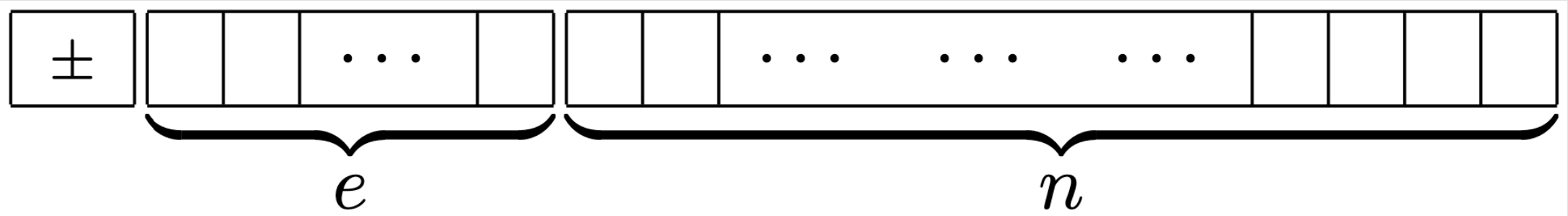
$$-1^{1} \times 2^{4} \times (2^{0} + 0.697265625) = -27.15625$$

Showing the binary representation of floats

```
import numpy as np
import struct
def float_to_binary(f):
    if type(f) == np.float16:
        nb_bits, s = 16, '<H'
    elif type(f) == np.float32:
        nb_bits, s = 32, '<I'
    elif type(f) == np.float64:
        nb_bits, s = 64, '<Q'
    else:
        return
    return ('{:0'+str(nb_bits)+'b}').format(struct.unpack(s, f.tobytes())[0])

print(float_to_binary(np.float16(-27.15625)))
## 1100111011001010
print(float_to_binary(np.float32(-27.15625)))
## 11000001110110010100000000000000
print(float_to_binary(np.float64(-27.15625)))
## 1100000000111011001010000000000000000000000000000000000000000000
```

Representation of real numbers



Whatever the size of the word, there is only a limited number of bits to store **e** and **n**. As a result it is ***impossible to represent all real numbers exactly***.

Computations involving real numbers follow the rules of **floating-point arithmetic**. Such computations generate ***rounding errors***.

Representation of real numbers

https://docs.python.org/3/tutorial/floatingpoint.html

```
1 + 1 + 1 == 3
## True
0.1 + 0.1 == 0.2
## True
0.1 + 0.1 + 0.1 == 0.3
## False
```

```
print(0.1)
## 0.1
print(format(0.1, '.50g'))
## 0.10000000000000000555111512312578270211815834045410
```

Representation of real numbers

Some real numbers can be stored exactly

$$\mathbf{0.125 = 1 \times 2^{-3}} \qquad \mathbf{0.1015625 = 13 \times 2^{-7} = (1 + 2^{-1} + 2^{-3}) \times 2^{-4}}$$

Most real numbers cannot

```
(0.1).as_integer_ratio()
## (3602879701896397, 36028797018963968)
```

$$0.1 \approx 3602879701896397 \times 2^{-55} = 0.1000000000000000055511151231\ldots$$

Python floats

For real numbers, Python uses 64 bits (IEEE 754 double-precision standard)

▷ 1 bit for the sign (positive / negative)

▷ 11 for the exponent **e**. The range is [-1022, 1023]

▷ 52 bits for the significant digits **n**

It can represent real numbers from $\pm\mathbf{2.23}\times\mathbf{10}^{-\mathbf{308}}$ to $\pm\mathbf{1.80}\times\mathbf{10}^{\mathbf{308}}$

Consider the absolute value x of the result of an expression:

▷ If x is smaller than $m = 2.23\times10^{-308}$, we get an ***underflow***

▷ If x is larger than $M = 1.80\times10^{308}$, we get an ***overflow***

NumPy floats

```
print(np.finfo(np.float32))
## Machine parameters for float32
## ---------------------------------------------------------------
## precision =   6   resolution = 1.0000000e-06
## machep =    -23   eps =        1.1920929e-07
## negep =     -24   epsneg =     5.9604645e-08
## minexp =   -126   tiny =       1.1754944e-38
## maxexp =    128   max =        3.4028235e+38
## nexp =        8   min =        -max
## smallest_normal = 1.1754944e-38   smallest_subnormal = 1.4012985e-45
## ---------------------------------------------------------------
print(np.finfo(np.float64))
## Machine parameters for float64
## ---------------------------------------------------------------
## precision =  15   resolution = 1.0000000000000001e-15
## machep =    -52   eps =        2.2204460492503131e-16
## negep =     -53   epsneg =     1.1102230246251565e-16
## minexp =  -1022   tiny =       2.2250738585072014e-308
## maxexp =   1024   max =        1.7976931348623157e+308
## nexp =       11   min =        -max
## smallest_normal = 2.2250738585072014e-308   smallest_subnormal = 4.9406564584124654e-324
## ---------------------------------------------------------------
```

Machine precision

The **machine precision** of a floating point arithmetic is defined as the smallest positive number ε_1 such that $\text{float}(1+\varepsilon_1) > 1$

The machine precision can be computed as follows

```
e = 1
while 1+e > 1:
    e = e/2
e = 2*e
print(e)
## 2.22044604925031e-16
```

or obtained by

```
np.spacing(1) # machine precision, same as np.finfo(float).eps
## 2.22044604925031e-16
```

Float rounding

Mathematically, additions and multiplications are associative

```
(a+b)+c = a+(b+c)
(a*b)*c = a*(b*c)
```

⚠ Floating point computation is **NOT** associative

Set $e = \varepsilon_1/2$

```
float((1+e)+e) == 1
float((1+(e+e)) > 1
```

Float rounding

⚠ The gap between two exact floats grows with the size of the float

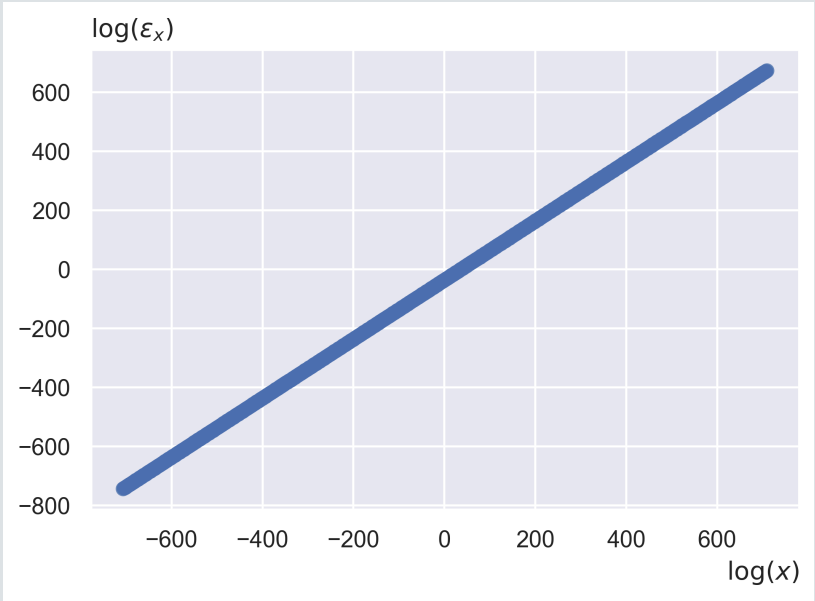
```
import numpy as np
np.spacing(10**(-10))
## 1.2924697071141057e-26
np.spacing(10**(-5))
## 1.6940658945086007e-21
np.spacing(1)
## 2.220446049250313e-16
np.spacing(10**5)
## 1.4551915228366852e-11
np.spacing(10**10)
## 1.907348632812e-06
```

Float rounding

Let ε_x be the precision gap at the real number x (`np.spacing(x)`).
For 64-bit floats, one has approximately:

$$\varepsilon_x \approx \mathrm{abs}(x) \times 1.5716 \times 10^{-16}$$

```
import numpy as np
from sklearn.linear_model import LinearRegression
x = np.power(10.0,np.arange(-307,309))
y = np.spacing(x)
logx = np.log(x).reshape(-1, 1)
reg = LinearRegression().fit(logx, np.log(y))
print(( reg.coef_ , np.exp(reg.intercept_) ))
## (array([0.99999942]), 1.5715891330946213e-16)
```



16-bit float spacing

```
import numpy as np
f16info = np.finfo(np.float16)
print("min = "+str(f16info.min)+", max = "+
      str(f16info.max)+", eps = "+str(f16info.eps))
## min = -65500.0, max = 65500.0, eps = 0.000977
```

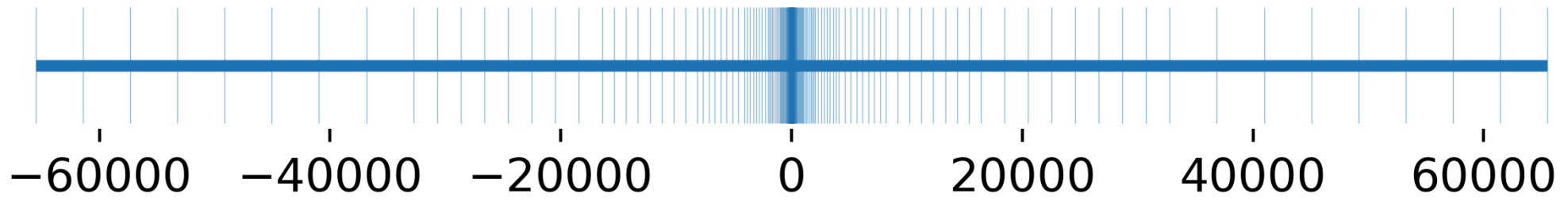
▷ The 16-bit float type can store 63487 unique exact real numbers.

▷ The representable real numbers **are not equally spaced**: the grid becomes coarser when moving to larger numbers.

16-bit float spacing

```
f16 = np.zeros(63487, dtype=np.float16)
f16[0] = np.finfo(np.float16).min
for i in np.arange(63487-1):
    f16[i+1] = f16[i] + np.spacing(f16[i],dtype=np.float16)

iselect=np.arange(-1,63488,128); iselect[0]=0;iselect[-1]=63486
for i in iselect: # only display one float every 128 floats
    ax.vlines(f16[i],-1,1,linewidth=0.25, alpha=0.5)
```



16-bit float spacing

Floats outside the grid are rounded to the closest exact float.

```
np.spacing([1024,2103],dtype=np.float16)
## array([1., 2.], dtype=float16)
v = np.zeros(4,dtype=np.float16)
v[0] = 1024.5
v[1] = 1078.5
v[2] = v[0] + v[1]
v[3] = 1024.5 + 1078.5
print((v[0],v[1]))
## (1024.0, 1078.0)
print((v[2], 1024.5 + 1078.5, v[3]))
## (2102.0, 2103.0, 2104.0)
```

Float rounding: geometric series

Consequently, computing a sum of floats $\sum_{i=1}^{n} x_i$ might give different results depending on the order of the sequence $x_i, i = 1, \ldots, n$.

```
import numpy as np
s1 = s2 = 0.0
powers = np.arange(-10.0,10.0)
for p in powers:
    s1 += 10**p
for p in np.flipud(powers):
    s2 += 10**p
print(s1)
## 1111111111.1111112
print(s2)
## 1111111111.1111107
```

Float rounding: geometric series

The exact result is $\sum_{p=-10}^{9} 10^p = (10^{10} - 10^{-10})/9$

```
print((10**10-10**(-10))/9)
## 1111111111.1111112
from decimal import Decimal
ten = Decimal(10)
(ten**ten-(ten**Decimal(-10)))/Decimal(9)
## Decimal('1111111111.111111111')
```

🖋 Summing from smaller to larger numbers generates less rounding error

The **`decimal`** package provides arbitrary-precision floating-point numbers, but is not compatible with NumPy https://docs.python.org/3/library/decimal.html

Float rounding: harmonic series

The harmonic series is known to diverge

$$\sum_{n=1}^{\infty} \frac{1}{n} = \infty$$

Computing this sum numerically will produce a finite result, as the number of terms has to be truncated to a finite number. What might be surprising is that this sum is numerically guaranteed to remain below 34. The reason is that this sum will stop growing numerically whenever n is such that

$$\frac{1}{n} < \varepsilon_1$$

Float rounding: harmonic series

More precisely, the harmonic number $H_n := \sum_{k=1}^{n} \frac{1}{k}$ stops growing as soon as

$$\frac{1}{n+1} < \varepsilon_{H_n}$$

Using the classical approximation $H_n \approx \log(n) + \gamma$, where $\gamma = 0.577215\ldots$ is the Euler-Mascheroni constant, one can estimate the number of terms n^* and the partial sum H_{n^*} such that the harmonic number stops growing numerically.
For 64-bit floats, the sum is guaranteed to remain below **34**. For 32-float the numerical sum is lower than **15**, and for 16-bit floats it is lower than **7**.

Recommendations

▷ Avoid comparisons between floats, or use **`isclose()`**

```
from math import isclose
isclose( 0.1 + 0.1 + 0.1 , 0.3 )
## True
```

▷ It is more accurate to sum floats from *smallest to largest* (in absolute value). One can also use accurate summation algorithms such as the Møller-Kahan summation algorithm.

▷ Use 64-bit floats instead of 32-bit floats. The package **`decimal`** provides arbitrary-precision floating-point numbers, but is not compatible with NumPy.

2.3 Numerical differentiation

Approximation of derivatives

Let $f : \mathbb{R} \to \mathbb{R}$ be a continuous and differentiable real function, with continuous derivative $f^{(1)}$. By definition, the derivative of f at the point $x \in \mathbb{R}$ is defined by

$$f^{(1)}(x) := \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

The derivative $f^{(1)}$ might be known explicitly if f has a simple analytical form. Nevertheless, most of the time the derivative $f^{(1)}$ is not known explicitly, and has to be approximated numerically. One can obtain an approximation of $f^{(1)}(x)$ by considering "small" values for h instead of letting it go to zero.

Question: **How to choose h** to get a good approximation of $f^{(1)}(x)$ when working with floating-point arithmetic?

Approximation of derivatives

Suppose that f can be differentiated $n+1$ times. Let $f^{(n)}$ be the n-th derivative. The Taylor expansion of f around x reads

$$f(x+h) = f(x)+hf^{(1)}(x)+\frac{h^2}{2}f^{(2)}(x)+\frac{h^3}{6}f^{(3)}(x)+\ldots+\frac{h^n}{n!}f^{(n)}(x)+\frac{h^{n+1}}{(n+1)!}f^{(n+1)}(\xi)$$

where $\xi \in [x, x+h]$.
We know that ξ exists but its exact value is not known.
A Taylor series approximates a function with a degree of precision equal to the remainder term $\frac{h^{n+1}}{(n+1)!}f^{(n+1)}(\xi)$.

Forward difference

Consider a first-order Taylor expansion with exact remainder

$$f(x+h) = f(x) + hf^{(1)}(x) + \frac{h^2}{2} f^{(2)}(\xi)$$

with $\xi \in [x, x+h]$ from which we get

$$f^{(1)}(x) = \frac{f(x+h) - f(x)}{h} - \frac{h}{2} f^{(2)}(\xi)$$

This expression for $f^{(1)}(x)$ is composed of two parts:
the **approximation of the derivative** and the **truncation error**.
This approximation is called **forward-difference** approximation.

Backward difference

Consider the alternative Taylor expansion

$$f(x-h) = f(x) - hf^{(1)}(x) + \frac{h^2}{2} f^{(2)}(\xi)$$

with $\xi \in [x-h, x]$ (h was replaced by $-h$). We get the approximation

$$f^{(1)}(x) = \frac{f(x) - f(x-h)}{h} + \frac{h}{2} f^{(2)}(\xi)$$

This is called the **backward-difference** approximation.

Centered difference

Consider the two Taylor expansions

$$f(x+h) = f(x) + hf^{(1)}(x) + \frac{h^2}{2}f^{(2)}(x) + \frac{h^3}{6}f^{(3)}(\xi_+) \quad \text{with } \xi_+ \in [x, x+h]$$

$$f(x-h) = f(x) - hf^{(1)}(x) + \frac{h^2}{2}f^{(2)}(x) - \frac{h^3}{6}f^{(3)}(\xi_-) \quad \text{with } \xi_- \in [x-h, x]$$

The difference between the two is equal to

$$f(x+h) - f(x-h) = 2hf^{(1)}(x) + \frac{h^3}{6}(f^{(3)}(\xi_+) + f^{(3)}(\xi_-))$$

Centered difference

If $f^{(3)}$ is continuous, then there exists $\xi \in [\xi_-, \xi_+]$ such that

$$f^{(3)}(\xi) = \frac{f^{(3)}(\xi_+) + f^{(3)}(\xi_-)}{2}$$

This gives

$$f^{(1)}(x) = \frac{f(x+h) - f(x-h)}{2h} - \frac{h^2}{6} f^{(3)}(\xi)$$

This is called the **centered-difference** approximation of $f^{(1)}(x)$. It is **more precise** than both the forward difference and backward difference, since the truncation error is of order $\mathcal{O}(h^2)$.

Approximating second-order derivatives

Computing the sum $f(x+h) + f(x-h)$

$$f(x+h) = f(x) + hf^{(1)}(x) + \frac{h^2}{2} f^{(2)}(x) + \frac{h^3}{6} f^{(3)}(\xi_+) \quad \text{with } \xi_+ \in [x, x+h]$$

$$f(x-h) = f(x) - hf^{(1)}(x) + \frac{h^2}{2} f^{(2)}(x) - \frac{h^3}{6} f^{(3)}(\xi_-) \quad \text{with } \xi_- \in [x-h, x]$$

we get

$$f^{(2)}(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} - \frac{h}{6}(f^{(3)}(\xi_+) - f^{(3)}(\xi_-))$$

Partial derivatives

Given a function of two variables $f(x, y)$, the centered-difference approximation of the partial derivative f_x of f with respect to x is

$$f_x := \frac{\partial f}{\partial x} = \frac{f(x + h_x, y) - f(x - h_x, y)}{2h_x}$$

where h_x is the mesh used in the domain of x.

Partial derivatives

Computing the centered difference of the partial derivative f_x with respect to y gives

$$f_{xy} := \frac{\partial^2 f}{\partial x \partial y} = \frac{\frac{f(x+h_x, y+h_y) - f(x-h_x, y+h_y)}{2h_x} - \frac{f(x+h_x, y-h_y) - f(x-h_x, y-h_y)}{2h_x}}{2h_y}$$

where h_y is the mesh used in the domain of y. This simplifies into the cross-derivative approximation

$$f_{xy} = \frac{f(x+h_x, y+h_y) - f(x-h_x, y+h_y) - f(x+h_x, y-h_y) + f(x-h_x, y-h_y)}{4h_x h_y}$$

How to choose h?

How to best choose h taking into account the constraints and limits of floating-point arithmetic?

We want to choose h as small as possible to reduce the **truncation error**. However we also need to consider the **rounding error**.

If h is *too small*, we might get

```
float(x+h)==float(x)
```

Even if it is not the case, we might still get

```
float(f(x+h))==float(f(x))
```

if the function f varies slowly.

Truncation error for forward-difference

Let $f_h^{(1)}(x)$ be the approximation of $f^{(1)}(x)$ using the mesh size h

$$f_h^{(1)}(x) = \frac{f(x+h) - f(x)}{h}$$

If we suppose that $f^{(2)}$ is bounded for all x in the domain of application:

$$|f^{(2)}(x)| \leq M$$

then we get the following bound for the **truncation error** $-\frac{\mathbf{h}}{\mathbf{2}}\mathbf{f}^{(\mathbf{2})}(\xi)$

$$|f_h^{(1)}(x) - f^{(1)}(x)| \leq \frac{h}{2}M$$

How to choose h?

The truncation error bound $hM/2$ suggests that we can reach any arbitrary precision by choosing h sufficiently small. However, in floating-point arithmetic it is not possible to evaluate $f_h^{(1)}(x)$ exactly, due to **float rounding errors**.

Suppose that the rounding error for evaluating $f(x)$ has the following bound

$$|\mathrm{float}(f(x)) - f(x)| \leq \varepsilon$$

Then the rounding error for a forward-difference approximation is bounded by

$$\left|\mathrm{float}\left(f_h^{(1)}(x)\right) - f_h^{(1)}(x)\right| = \left|\frac{\mathrm{float}\left(f(x+h)\right) - f(x+h)}{h} - \frac{\mathrm{float}\left(f(x)\right) - f(x)}{h}\right| \leq \frac{2\varepsilon}{h}$$

How to choose h?

Combining both error bounds, we obtain

$$\left|\mathrm{float}\left(f_h^{(1)}(x)\right) - f^{(1)}(x)\right| \leq \frac{2\varepsilon}{h} + \frac{M}{2}h$$

One needs to find a compromise between reducing the **truncation error** and reducing the **rounding error**

The precision we can achieve is **limited**

▷ Note that h can be represented without rounding error if we choose it as a power of 2

Optimal h for forward difference

We can minimize the bound $b(h) = \frac{2\varepsilon}{h} + \frac{hM}{2}$ as a function of h:

$$b^{(1)}(h) = -\frac{2\varepsilon}{h^2} + \frac{M}{2} = 0 \quad \Leftrightarrow \quad \mathbf{h = 2\sqrt{\frac{\varepsilon}{M}}}$$

If we assume that M is of order 1 and that ε is the 64-bit machine precision $\varepsilon_1 = 2.22 \times 10^{-16} = 2^{-52}$ we get

$$\mathbf{h = 2.98 \times 10^{-8} = 2^{-25}}$$

```
import numpy as np
h=2*np.sqrt(np.spacing(1))
print( ( h , h==2**(-25) ) )
## (2.9802322387695312e-08, True)
```

Optimal h for centered difference

For centered difference approximation, if we assume that $|f^{(3)}(x)| \leq M$ for all x in the domain, the total error bound is $b(h) = \frac{\varepsilon}{h} + \frac{h^2 M}{6}$

$$b^{(1)}(h) = -\frac{\varepsilon}{h^2} + \frac{Mh}{3} = 0 \quad \Leftrightarrow \quad \mathbf{h} = \left(\frac{\mathbf{3}\varepsilon}{\mathbf{M}}\right)^{\mathbf{1/3}}$$

If M is of order 1 and $\varepsilon = \varepsilon_1$ we get

$$\mathbf{h \approx 8.7 \times 10^{-6} \approx 1.14 \times 2^{-17}}$$

```
import numpy as np
h=(3*np.spacing(1))**(1/3)
print( ( h , h/(2**(-17)) ) )
## (8.733476581980381e-06, 1.144714242533325)
```

Example

To illustrate the influence of h on the quality of the numerical approximation of a derivative function, consider the function

$$f(x) = \cos(x^x) - \sin(e^x)$$

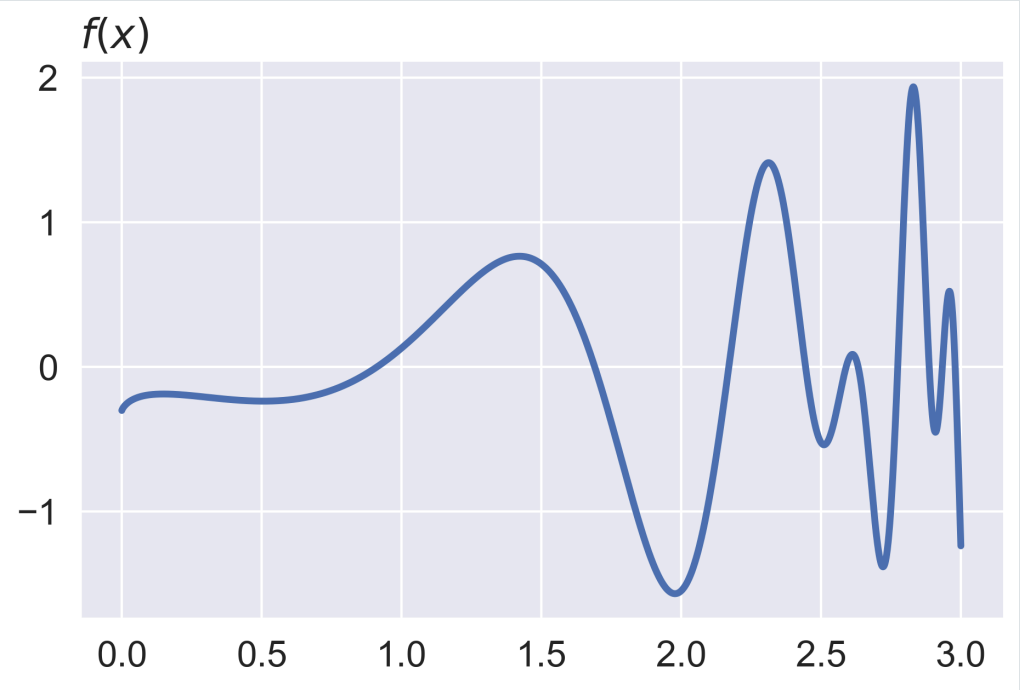
Its derivative can be computed explicitly

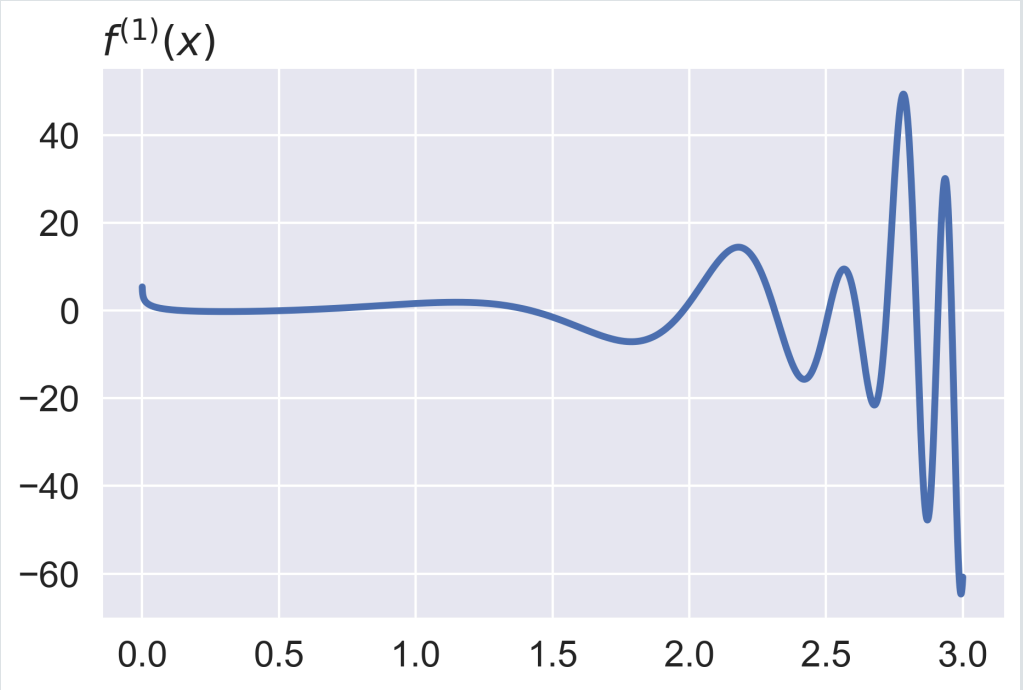
$$f^{(1)}(x) = -\sin(x^x)x^x(\log(x) + 1) - \cos(e^x)e^x$$

Higher-order derivatives can also be computed explicitly

Example

These two functions $f(x)$ and $f^{(1)}(x)$ look like this





Example

Let $f_h^{(1)}(x)$ be the finite-difference approximation of $f^{(1)}(x)$. To measure the quality of the approximation, we can compute the relative error between the two

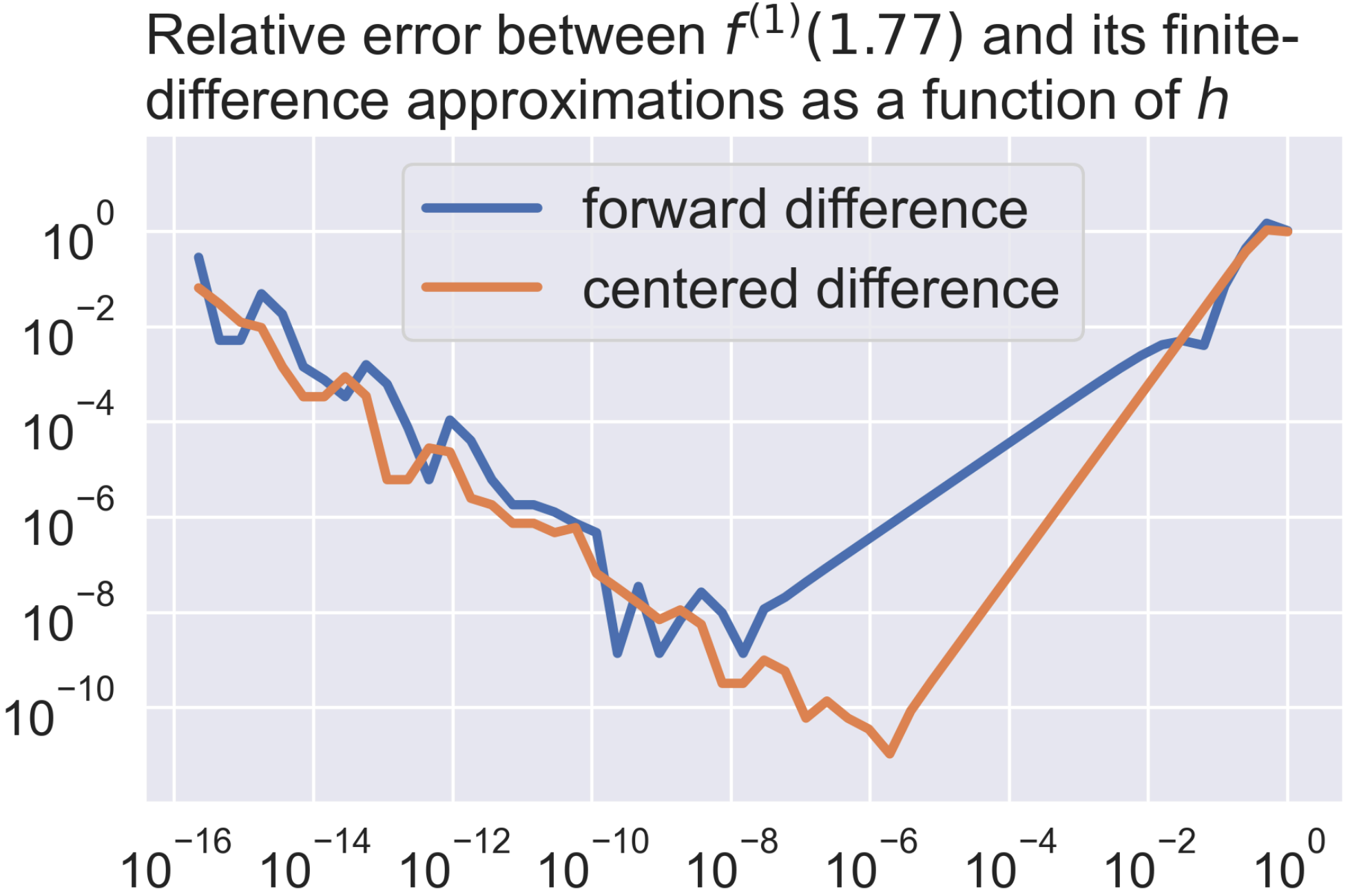
$$\left| \frac{f_h^{(1)}(x) - f^{(1)}(x)}{f^{(1)}(x)} \right|$$

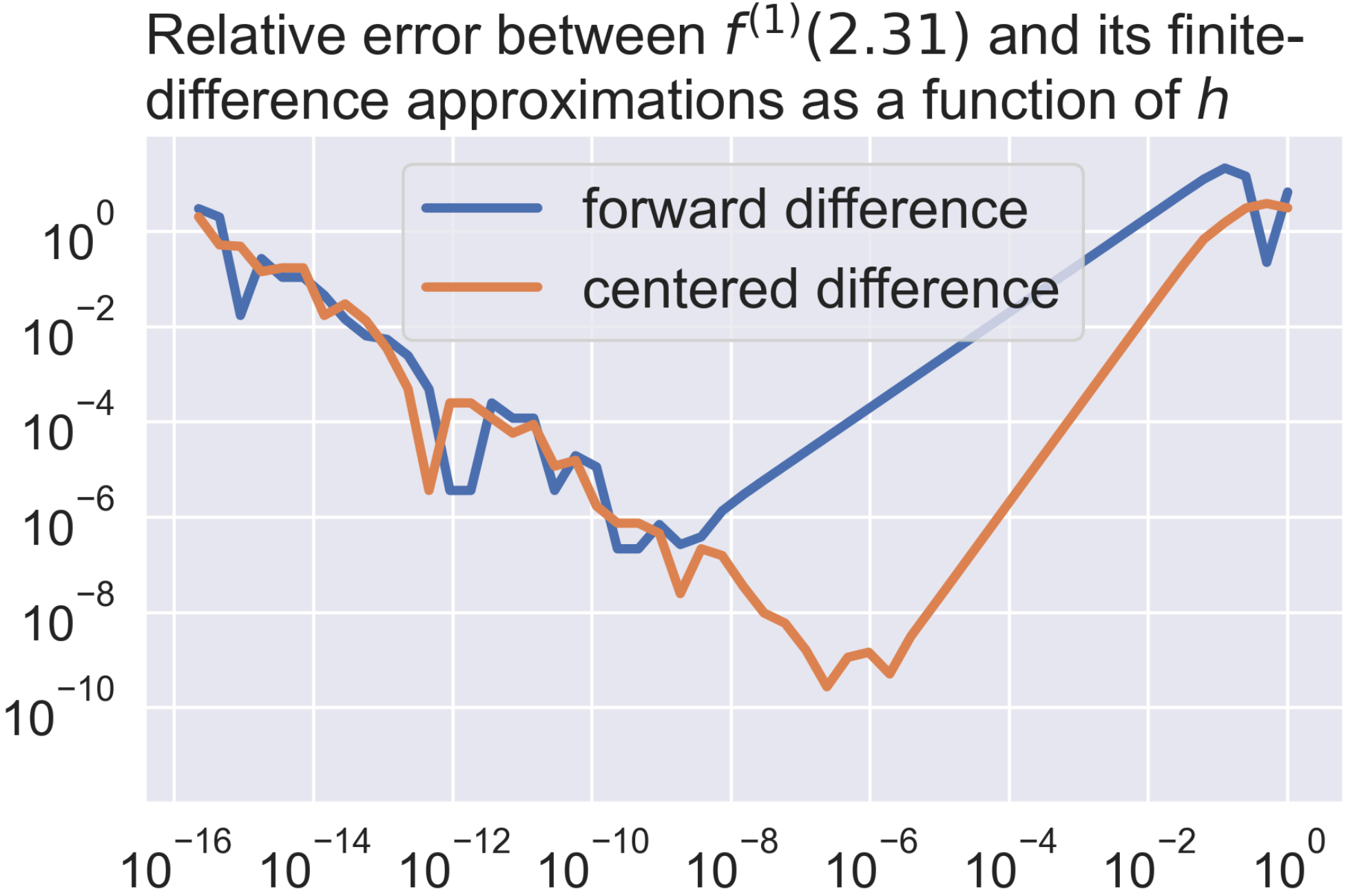
⚠ In general, the exact derivative $f^{(1)}(x)$ is **unknown**

We investigate the accuracy of the forward-difference and centered-difference approximations at the points $x = 1.77$ and $x = 2.31$

```
### Computing the finite-difference approximations of f'(x)
import numpy as np
x = 1.77
h = 2.0**np.arange(-52,1,1)
def f(x):
    return np.cos(x**x)-np.sin(np.exp(x))
def f1(x):
    return -np.sin(x**x)* x**x * (np.log(x)+1) - np.cos(np.exp(x)) * np.exp(x)
(f_x, f1_x, f_x_plus_h, f_x_minus_h) = (f(x), f1(x), f(x+h), f(x-h))
f1_forward_diff = (f_x_plus_h - f_x) / h ### forward-difference approximation
f1_centered_diff = (f_x_plus_h - f_x_minus_h) / (2*h) ### centered-difference approximation

### Plotting the relative error of the finite-difference approximations
fig, ax = plt.subplots(1)
ax.loglog(h,np.abs((f1_forward_diff-f1_x)/f1_x), label='forward difference', linewidth=3)
ax.loglog(h,np.abs((f1_centered_diff-f1_x)/f1_x), label='centered difference', linewidth=3)
ax.set_xticks(10.0**np.arange(-16,1,2))
ax.set_yticks(10.0**np.arange(-10,1,2))
ax.set_ylim([10**(-12),100.0])
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.set_title('Relative error between $f^{(1)}('+str(x)+')$ and its \nfinite difference approximatio
ax.legend(loc='upper center')
fig.tight_layout(pad=0)
plt.subplots_adjust()
plt.show()
```


Relative error between $f^{(1)}(1.77)$ and its finite-difference approximations as a function of h
forward difference
centered difference
10^{0}
10^{-2}
10^{-4}
10^{-6}
10^{-8}
10^{-10}
10^{-16} 10^{-14} 10^{-12} 10^{-10} 10^{-8} 10^{-6} 10^{-4} 10^{-2} 10^{0}


Relative error between $f^{(1)}(2.31)$ and its finite-difference approximations as a function of h
forward difference
centered difference
10^{0}
10^{-2}
10^{-4}
10^{-6}
10^{-8}
10^{-10}
10^{-16} 10^{-14} 10^{-12} 10^{-10} 10^{-8} 10^{-6} 10^{-4} 10^{-2} 10^{0}

Observations

▷ There is an optimal h^* which best balances the **truncation error** and the **rounding error**

▷ If $h > h^*$, the **truncation error** dominates

▷ If $h < h^*$, the **rounding error** dominates

▷ The centered difference approximation is better than the forward difference and backward difference approximations, due to its lower **truncation error**

▷ The optimal h^* is lower for forward/backward difference than for centered difference because of their greater **truncation error**

▷ The higher $\left|f^{(2)}(x)\right|$ (for forward/backward difference) and $\left|f^{(3)}(x)\right|$ (for centered difference), the higher the optimal relative error and the lower the optimal h^*, due to higher **truncation error**

2.4 Measuring approximation errors

Absolute error

How to measure the error between a value x and an approximation $\hat{x}$?

The **absolute error** between x and $\hat{x}$ is simply

$$|\hat{x} - x|$$

If $x = 2$ and $\hat{x} = 3$, then the absolute error is 1.
If $x = 10^9$ and $\hat{x} = 10^9 + 1$, the absolute error is also 1.
However, the error can be deemed "smaller" than in the previous example.

Relative error

The **relative error** between a value x and its approximation $\hat{x}$ is defined by

$$\frac{|\hat{x} - x|}{|x|}$$

In the first example $x = 2$ and $\hat{x} = 3$, the relative error is 0.5.
In the second example $x = 10^9$ and $\hat{x} = 10^9 + 1$, the relative error is 10^{-9}, which is much smaller.

▷ Note that the relative error is only defined if $x \neq 0$.
If $x = 0$, then the absolute error should be used instead.

Combining absolute and relative error

One can combine the absolute and relative error into the following error measure

$$\frac{|\hat{x} - x|}{|x| + 1}$$

- ▷ It avoids the problems of the relative error when x is near zero
- ▷ It behaves like the relative error when $|x| \gg 1$
- ▷ It behaves like the absolute error when $|x| \ll 1$

2.5 Numerical instability and ill-conditioning

Numerical instability and ill-conditioning

If the quality of a numerical solution is not acceptable, it is important to distinguish between the following two situations:

▷ Numerical instability

Errors are amplified by the algorithm

▷ Ill-conditioned problem

Small perturbations of the data generate large changes in the solution

Roots of quadratic equation

As an example, consider the numerical solution of the classical quadratic equation

$$ax^2 + bx + c = 0$$

The analytical solutions are the two quadratic formulas

$$x_1 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}, \quad x_2 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

Define the discriminant $\Delta = b^2 - 4ac$.

Roots of quadratic equation: direct algorithm

One can directly transcribe the analytical solution into a direct algorithm. For $a = 1$, $c = 2$ and using 64-bit floats, the numerical solution x_2 for three different values of $b \in \{5.2123, 121.23, 1212.3\}$ are given by

```
import numpy as np
a = 1.0
b = np.array([5.2123, 121.23, 1212.3])
c = 2.0
sqrt_delta = np.sqrt(b*b-4*a*c)
x1 = (-b-sqrt_delta)/(2*a)
x2 = (-b+sqrt_delta)/(2*a)
print(x2)
## [-0.4170822  -0.01649981 -0.00164976]
```

Roots of quadratic equation: direct algorithm

To pinpoint the problem with this algorithm, let us use 16-bit floats instead

```
import numpy as np
a16 = np.array([1.0],dtype='float16')
b16 = np.array([5.2123, 121.23, 1212.3],dtype='float16')
c16 = np.array([2.0],dtype='float16')
sqrt_delta_f16 = np.sqrt(b16*b16-4.0*a16*c16)
print(sqrt_delta_f16)
## [  4.375 121.25      inf]
```

First, b^2 is outside the range of 16-bit floats for the third value $b = 1212.3$. For the sake of this experiment, let us compute $\sqrt{\Delta}$ with 64-bit floats, and then round the result into a 16-bit float.

Roots of quadratic equation: direct algorithm

```
sqrt_delta_f16 = np.sqrt(b*b-4.0*a*c).astype('float16')
x1_f16 = (-b16-sqrt_delta_f16)/(2*a16)
x2_f16 = (-b16+sqrt_delta_f16)/(2*a16)
print(sqrt_delta_f16)
print(x2_f16) # numerical estimation of x2
print(np.abs(x2_f16-x2)/np.abs(x2)) # relative error
```

```
## [   4.38  121.2  1212.  ]
## [-0.416   -0.03125  0.     ]
## [0.00255724 0.89396094 1.        ]
```

The relative error for the numerical estimation of x_2 is $\mathbf{89.4\%}$ for $b = 121.23$, and $\mathbf{100\%}$ for $b = 1212.3$

Roots of quadratic equation: direct algorithm

This is an example of **catastrophic cancellation**:
the difference $-b + \sqrt{\Delta}$ in the analytical solution of x_2 can be very inaccurate if the two numbers b and $\sqrt{\Delta}$ are relatively large.

⚠ There is a loss of significant digits when subtracting two large numbers

The direct algorithm amplifies the rounding error:
it is a **numerically unstable** algorithm.

⚠ This numerical instability also affects 64-bit floating-point arithmetic: it just requires a larger size of b for the problem to occur as well

Roots of quadratic equation: stable algorithm

Using the relation $x_1 x_2 = c/a$ (Viète's formula) can stabilize the algorithm

```
if b<0:
  x2_f16 = (-b16+sqrt_delta_f16)/(2*a16)
  x1_f16 = c16/(a16*x2_f16)
else:
  x1_f16 = (-b16-sqrt_delta_f16)/(2*a16)
  x2_f16 = c16/(a16*x1_f16)
print(x2_f16) # numerical estimation of x2
print(np.abs(x2_f16-x2)/np.abs(x2)) # relative error
```

```
## [-0.417   -0.0165  -0.00165]
## [2.15820519e-04 3.06749703e-04 5.91973937e-05]
```

Roots of quadratic equation: stable algorithm

Another approach: for large $b > 0$, use the following Taylor expansion

$$\sqrt{1-\varepsilon} = 1 - \frac{1}{2}\varepsilon - \frac{1}{8}\varepsilon^2 + o(\varepsilon^2), \quad \text{for } |\varepsilon| \ll 1$$

to obtain the following stable approximation of x_2 (set $\varepsilon = 4ac/b^2$)

$$x_2 = \frac{-b+\sqrt{b^2-4ac}}{2a} = \frac{-b+b\sqrt{1-\frac{4ac}{b^2}}}{2a} \approx -\frac{c}{b} - \frac{ac^2}{b^3}$$

```
x2_f16_taylor =  - c16/b16 - (a16/b16)*(c16/b16)**2
print(x2_f16_taylor) # numerical estimation of x2
print(np.abs(x2_f16_taylor-x2)/np.abs(x2)) # relative error
```

```
## [-0.412   -0.0165  -0.00165]
## [1.19228952e-02 3.06749703e-04 5.91973937e-05]
```

Linear system

Consider the linear system of equations $Ax = b$ where

$$A = \begin{bmatrix} .780 & .563 \\ .913 & .659 \end{bmatrix}, \quad b = \begin{bmatrix} .217 \\ .254 \end{bmatrix}$$

The exact solution is $x = \begin{bmatrix} 1 & -1 \end{bmatrix}^\top$ (⚠ In general it is not known)

```
import numpy as np
A = np.array([[.780, .563], [.913, .659]])
b = np.array([.217, .254])
x = np.linalg.solve(A, b)
print( ( x , np.allclose(np.dot(A, x), b) ) )
## (array([ 1., -1.]), True)
```

Linear system

Now consider the small perturbation matrix

$$\Delta A = \begin{bmatrix} .001 & .001 \\ -.002 & -.001 \end{bmatrix}$$

and the perturbed system $(A + \Delta A)(x + \Delta x) = b$. The new solution is

```
import numpy as np
A = np.array([[.780, .563], [.913, .659]])
Delta_A = np.array([[.001, .001], [-.002, -.001]])
b = np.array([.217, .254])
new_x = np.linalg.solve(A+Delta_A, b)
print(new_x)
## [-5.          7.30851064]
```

Ill-conditioned problem

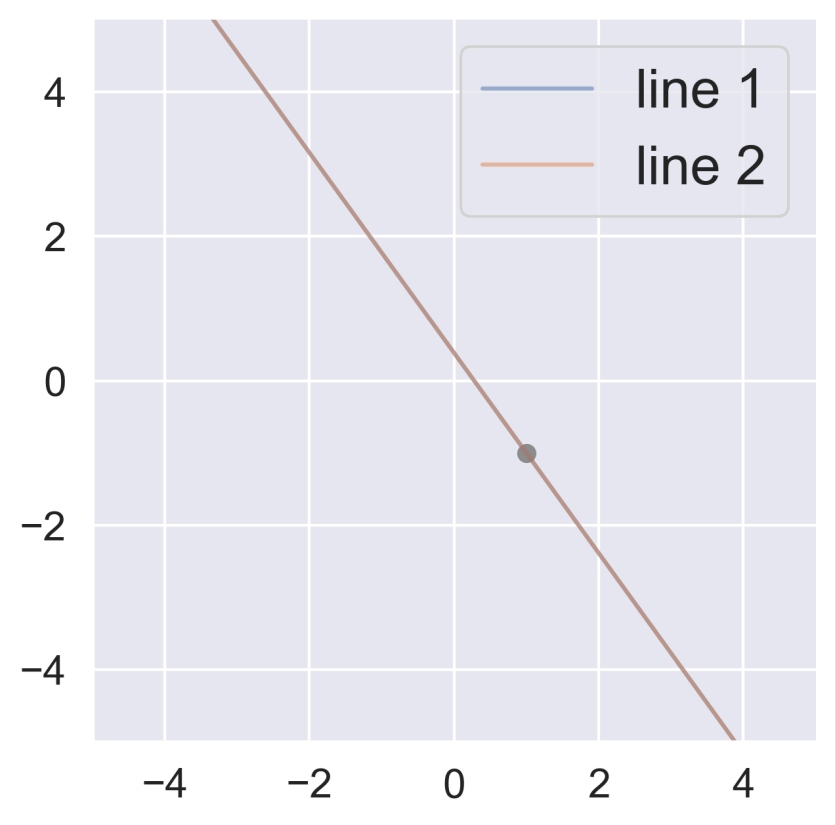
```
input_relative_change = np.mean(np.abs(Delta_A/A))
output_relative_change = np.mean(np.abs((new_x-x)/x))
print((input_relative_change , output_relative_change))
print(output_relative_change / input_relative_change)
```

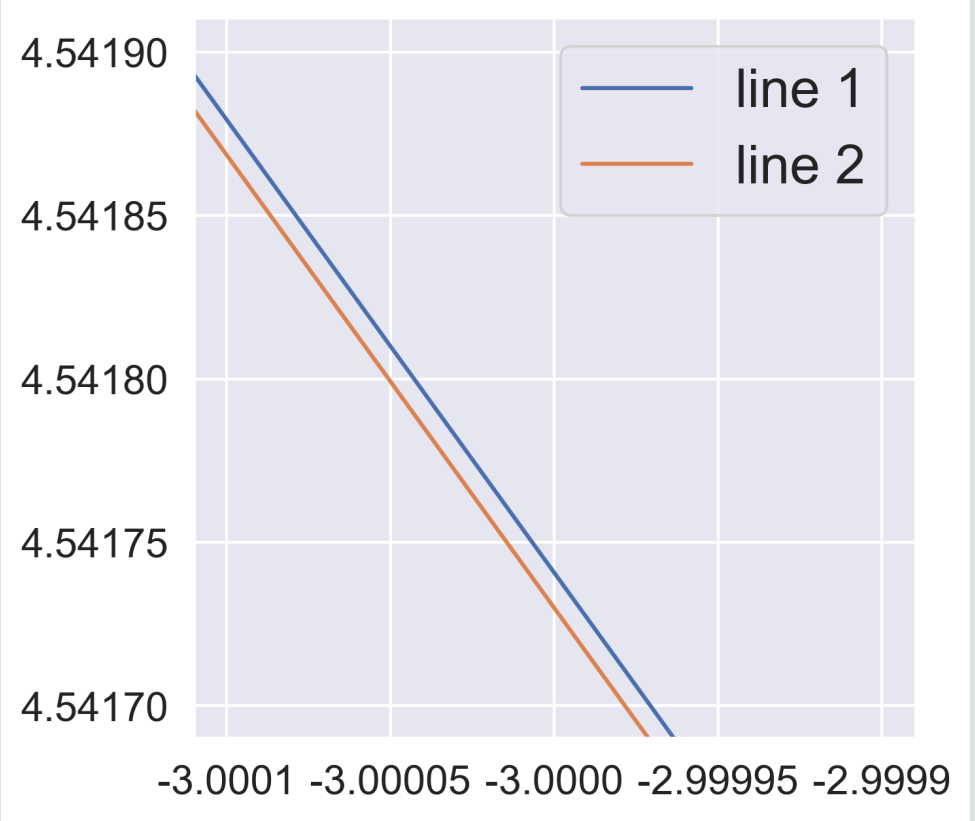
```
## (0.0016915703507545611, 7.154255319977163)
## 4229.35724593769
```

A relative change of less than **0.2%** of the input values causes a relative change of more than **700%** of the output values! The relative change ratio is greater than **4000**. This is an example of an **ill-conditioned problem**: small changes in the input values can cause large changes in the output values.

Solution of a linear system

The solution of the linear system $Ax = b$ corresponds to the intersection of the two lines $A_{11}x_1 + A_{12}x_2 = b_1$ and $A_{21}x_1 + A_{22}x_2 = b_2$





Condition number

In the general case, the (relative) **condition number** of a function f from $\mathbb{R}^n$ to $\mathbb{R}^n$ at an input point $x \in \mathbb{R}^n$ is

$$\operatorname{cond}(f, x) = \lim_{\varepsilon \to 0} \sup_{\|\Delta x\| \leq \varepsilon \|x\|} \frac{\|f(x + \Delta x) - f(x)\|}{\varepsilon \|f(x)\|}$$

where the Euclidean norm of any vector $a = (a_1, \ldots, a_n) \in \mathbb{R}^n$ is

$$\|a\| = \sqrt{\sum_{i=1}^{n} a_i^2}$$

Condition number

If f is differentiable at x, then an explicit expression for $\mathrm{cond}(f, x)$ can be obtained in terms of the Jacobian matrix $J(x) = (\partial f_i / \partial x_j) \in \mathbb{R}^{n \times n}$

$$\mathrm{cond}(f, x) = \frac{\|x\| \|J(x)\|}{\|f(x)\|}$$

where the norm of any matrix $A = (a_{i,j}) \in \mathbb{R}^{n \times n}$ is

$$\|A\| = \max_{x \in \mathbb{R}^n \setminus \{0\}} \frac{\|Ax\|}{\|x\|}$$

Condition number

If f is a differentiable scalar function then $J(x) = f^{(1)}(x)$ and

$$\mathrm{cond}(f, x) = |x f^{(1)}(x)/f(x)|$$

Examples: $\mathrm{cond}(\exp, x) = |x|$, $\mathrm{cond}(\log, x) = 1/|\log(x)|$

A problem is said to be **well conditioned** if the condition number is small and **ill conditioned** if the condition number is large.

If f is *well conditioned*, small changes in x produce *small* changes in f.
If f is *ill conditioned*, small changes in x can produce *large* changes in f.

Condition number of a matrix

⚠ In general the condition number is not known and estimating it is at least as hard as estimating $f(x)$.

If the problem is to solve a linear system $Ax = b$, the condition number of the problem corresponds to the **condition number of the matrix $\mathbf{A}$**, denoted as $\kappa(A)$. By definition

$$\kappa(A) = \|A\|\|A^{-1}\|$$

Note that the norm $\|A\| = \max_x \|Ax\|/\|x\|$ measures how much the mapping induced by A can stretch vectors.

Condition number of a matrix

Consider now how much the matrix A can shrink vectors

$$\min_x \frac{\|Ax\|}{\|x\|} = \min_y \frac{\|y\|}{\|A^{-1}y\|} = \frac{1}{\max_y \frac{\|A^{-1}y\|}{\|y\|}} = \frac{1}{\|A^{-1}\|}$$

As a result, the condition number $\kappa(A)$ is the ratio of the maximum to minimum stretching. One can show that it is equal to the ratio of the maximum singular value of A over the minimum singular value of A

$$\kappa(A) = \|A\|\|A^{-1}\| = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}$$

Condition number of a matrix

In Python, the condition number of a matrix can be computed with NumPy linear algebra functions (`numpy.linalg`)

```
import numpy as np
A = np.array([[.780, .563], [.913, .659]])
print(np.linalg.cond(A))
```

```
## 2193218.99965077
```

```
(U,S,V) = np.linalg.svd(A)
print(S[0]/S[1])
```

```
## 2193218.99965077
```

Condition number as error magnifier

If x is the solution of the linear system $Ax = b$ and $x + \Delta x$ is the solution of $A(x + \Delta x) = b + \Delta b$, then one can show that

$$\frac{\|\Delta x\|}{\|x\|} \leq \kappa(A) \frac{\|\Delta b\|}{\|b\|}$$

The quantity $\|\Delta b\|/\|b\|$ is the *relative* change in the right-hand side, and the quantity $\|\Delta x\|/\|x\|$ is the resulting *relative* change in the solution.

This inequality shows that changes in the right-hand side can cause changes $\kappa(A)$ times as large in the solution: the condition number is a relative error magnification factor.

Condition number as error magnifier

If x is the solution of the linear system $Ax = b$ and $x + \Delta x$ is the solution of $(A + \Delta A)(x + \Delta x) = b$, then one can show that

$$\frac{\|\Delta x\|}{\|x\|} \leq \frac{\kappa(A)}{1 - \kappa(A)\frac{\|\Delta A\|}{\|A\|}} \frac{\|\Delta A\|}{\|A\|}$$

under the assumption $\|A^{-1}\|\|\Delta A\| = \kappa(A)\|\Delta A\|/\|A\| < 1$, see proof in Chapter 7 of

[1] N. J. Higham. *Accuracy and stability of numerical algorithms*. 2nd ed. 2002.

Ill-conditioning vs. numerical instability

▷ As a rule of thumb, the perturbations of the inputs of the problem are amplified by a factor lower than or equal to the **condition number** when solving the problem.

▷ A high condition number (ill-conditioning) is a ***property*** of a problem; it is not cause by any truncation or rounding error.

▷ However, a large condition number can cause additional **float-rounding** errors when solving the problem. As a rule of thumb, if $\kappa(A) = 10^p$, then p digits are lost in floating-point arithmetic.

Remarks and recommendations

▷ The large variation of outputs of an ill-conditioned problem is inherent to the problem itself. The best approach is usually to modify the problem through **regularization**.

▷ In general, the properties of a problem are not known beforehand (this is why numerical approximations are needed). It is advisable to perform a **sensitivity analysis** of the inputs to detect any ill-conditioning, and test a large range of input values to detect any numerical instability.

▷ In finance, numerical instability and ill-conditioning can occur for example for *option pricing* or *portfolio allocation*. In the latter case, the algorithm might suggest large, impractical rebalancing decisions.

Concluding remarks

If we want to obtain sound numerical results with a computer, we need to make sure that we have:

- a **well-conditioned problem**
- a **numerically stable algorithm** (if executed with finite precision)
- a software which is a **good implementation** of the algorithm

A numerically stable algorithm will never be able to solve an ill-conditioned problem with more precision than the data contains.
Conversely, a numerically unstable algorithm may produce bad solutions even for well-conditioned problems.

2.6 Complexity of algorithms

Problem size

How to evaluate and compare the performance of different algorithms for solving the same problem?

First, we need to define the size of the problem. The size is generally expressed as roughly the number of elements the algorithm has to process (amount of data). One can usually find a parameter $\mathbf{n}$ which describes the size of the problem.

Examples:

- ▷ computing the dot product of two vectors of size $\mathbf{n}$ is a problem of size $\mathbf{n}$
- ▷ multiplying two $\mathbf{n} \times \mathbf{n}$ matrices is a problem of size $\mathbf{n} \times \mathbf{n}$

Execution time

One important criterion for comparing different algorithms is their **execution time**. In practice, the execution time is measured in terms of the number of **elementary operations** required by the algorithm to solve a problem of size $\mathbf{n}$. Elementary operations include additions, subtractions, multiplications and divisions.

Remark: if the elementary operations involve floats, they are often called **flops** (*floating-point operations*). A computer performance can then be measured in **FLOPS** (a.k.a. flop/s, for *floating-point operations per second*).

Computational complexity

As an illustration, consider two algorithms $\mathcal{A}_1$ and $\mathcal{A}_2$ for which the number of elementary operations is $C_{\mathcal{A}_1}(n) = \frac{1}{2}n^2$ and $C_{\mathcal{A}_2}(n) = 5n$ respectively. For $n > 10$, the algorithm $\mathcal{A}_2$ is faster than $\mathcal{A}_1$.

In practice the computing time matters for large values of n, therefore people report the order of the computational complexity of algorithms using the **big $\mathbf{O}$ notation**: the complexity of algorithm $\mathcal{A}_1$ is $\mathcal{O}(n^2)$ and the one of $\mathcal{A}_2$ is $\mathcal{O}(n)$.

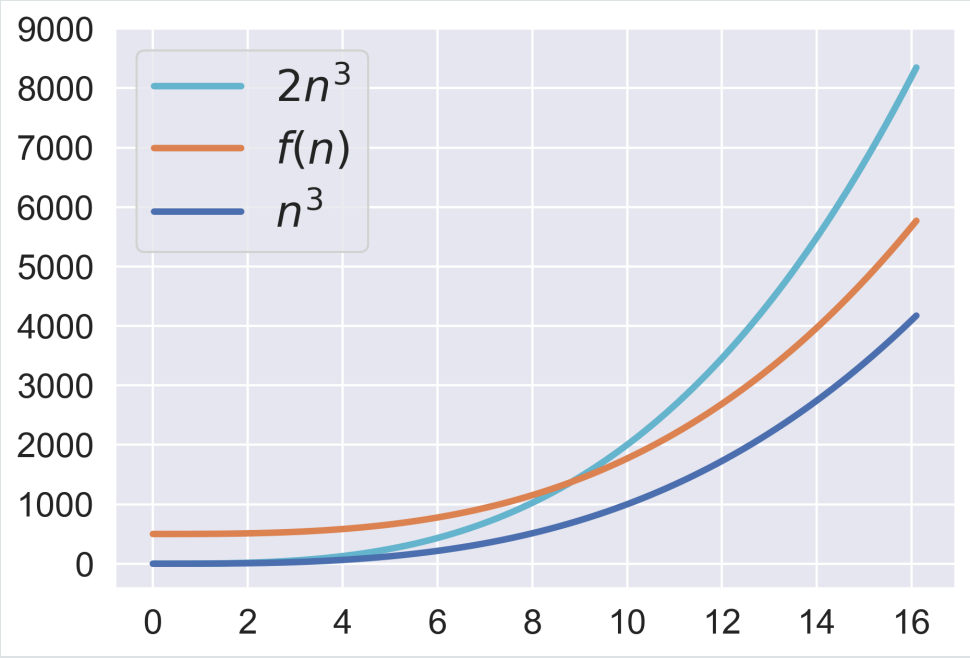
Another example: the computational complexity of an algorithm which requires $\frac{1}{3}n^3 + n^2 + \frac{2}{3}n$ elementary operations is of order $\mathcal{O}(n^3)$.

Complexity order $\mathcal{O}(.)$

A function $f(n)$ is $\mathcal{O}(g(n))$ if there exist constants c_0 and n_0 such that

$$|f(n)| \leq c_0 |g(n)| \quad \text{for all} \quad n \geq n_0$$

Example
$f(n) = \frac{5}{4}n^3 + \frac{1}{5}n^2 + 500$
$f(n) = \mathcal{O}(n^3)$ as
$|f(n)| < c_0 n^3$ for $n > n_0$
where $c_0 = 2$ and $n_0 = 9$



Complexity order $\mathcal{O}(.)$

Here is a list of classes of functions that are commonly encountered when analyzing algorithms. $c > 0$ is some arbitrary constant.

$\mathcal{O}(1)$	constant
$\mathcal{O}(\log(n))$	logarithmic
$\mathcal{O}((\log(n))^c)$	polylogarithmic
$\mathcal{O}(n)$	linear
$\mathcal{O}(n^2)$	quadratic
$\mathcal{O}(n^c)$	polynomial
$\mathcal{O}(c^n)$	exponential

Examples of computational complexity

▷ Sorting n numbers requires $\mathcal{O}(n\log(n))$ operations with the **quicksort** algorithm (which is the default sorting algorithm in NumPy). The **bubble sort** algorithm has a worst $\mathcal{O}(n^2)$ complexity.

▷ Multiplying two $n\times n$ matrices requires $\mathcal{O}(n^3)$ operations by direct matrix product. The matrix multiplication algorithm with best asymptotic complexity runs in $\mathcal{O}(n^{2.373})$. However, it is a **galactic algorithm**: the large computational constant makes it only worthwhile for matrices that are too large to handle on present-day computers.

▷ Inverting an $n\times n$ matrix requires $\mathcal{O}(n^3)$ operations by **Gauss-Jordan elimination**. A similar galactic algorithm requires $\mathcal{O}(n^{2.373})$ operations.

Classification of algorithms

The **computational complexity** is useful to compare the performance of algorithms. Two groups can be made:

▷ **Polynomial-time** algorithms: the complexity is a polynomial function of the problem size. These algorithms can be considered efficient. Examples: $\mathcal{O}(n^2)$, $\mathcal{O}(n^3)$, but also $\mathcal{O}(n\log(n))$ since $n \leq n\log(n) \leq n^2$ for $n > e$.

▷ **Non-polynomial-time** algorithms: the complexity is exponential or worse. These algorithms should generally be avoided except perhaps for problems with very small size n.

Other comparison criteria include the **space complexity** (a.k.a. memory complexity), which is the amount of memory necessary to execute the algorithm. Another criterion is the simplicity of coding.